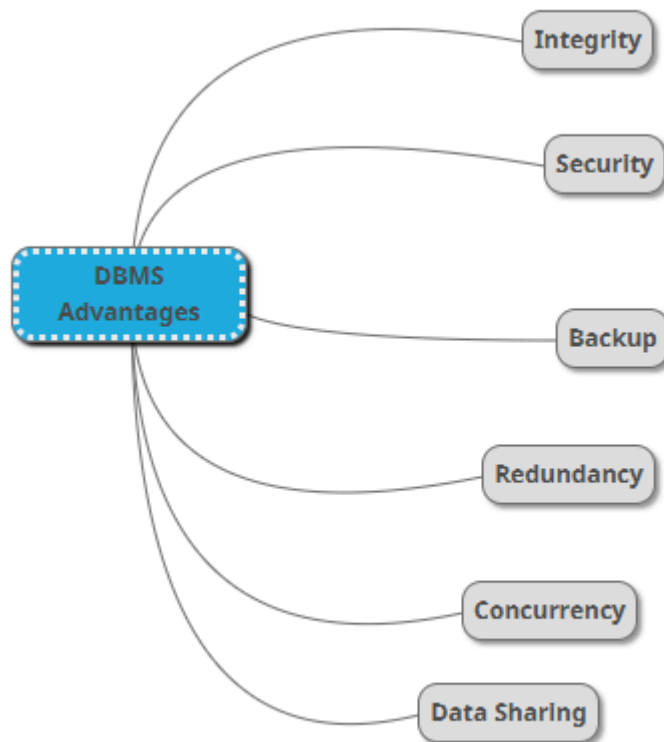


Database Course Documentation Report

1. Comparison Assignment: Flat File Systems vs Relational Databases

Feature	Flat File System	Relational Database
Structure	Stores data in a single file or multiple unrelated files without predefined format	Organizes data in structured tables with predefined schemas (rows and columns)
Data Redundancy	High; the same data might be repeated across files due to lack of relationships	Low; normalization techniques eliminate repetition through structured references
Relationships	Cannot represent relationships natively; relies on manual linking or duplication	Supports one-to-many and many-to-many relationships using primary and foreign keys
Example Usage	Simple data logging, text-based storage (CSV files, log files)	Business applications, e-commerce systems, CRM, financial databases
Drawbacks	No querying capability, lacks integrity checks, limited scalability	Requires DBMS setup and management, can be complex for small-scale needs

2. DBMS Advantages



- **Security:** Provides role-based access controls, user authentication, and data encryption.
 - **Integrity:** Enforces data validity through constraints, types, and validation rules.
 - **Backup:** Facilitates scheduled and automated backups to prevent data loss.
 - **Redundancy:** Reduces data duplication through normalization and central data storage.
 - **Concurrency:** Manages simultaneous data access by multiple users without conflicts.
 - **Data Sharing:** Allows different applications and users to access data concurrently while maintaining integrity.
-

3. Roles in a Database System

- **System Analyst:** Gathers requirements, analyzes system workflows, and aligns business processes with database solutions.
 - **Database Designer:** Develops logical and physical database models, ensuring efficient data storage and access paths.
 - **Database Developer:** Implements and optimizes queries, stored procedures, and data access layers within applications.
 - **Database Administrator (DBA):** Maintains database performance, monitors security, applies patches, and ensures backup/recovery operations.
 - **Application Developer:** Builds client-side and server-side applications that interact with the database to retrieve and manipulate data.
 - **Business Intelligence (BI) Developer:** Designs and develops dashboards, reports, and data visualization tools for decision support.
-

Additional Topics

Types of Databases

- **Relational Databases:** Based on table structure; strict schemas; uses SQL (MySQL, PostgreSQL, Oracle).
- **Non-Relational Databases:** Schema-less or flexible schema models; handles large and unstructured data (MongoDB - document-based, Cassandra - wide-column store).
- **Centralized Databases:** All data stored in a single location, often leading to easier control but limited fault tolerance.
- **Distributed Databases:** Data is distributed across multiple physical locations, enabling high availability and fault tolerance.
- **Cloud Databases:** Hosted in the cloud with features like auto-scaling, disaster recovery (AWS RDS, Azure SQL, Google Cloud Spanner).

Cloud Storage and Databases

- **Definition:** Cloud storage is a model where data is stored on remote servers and accessed over the internet.
- **Relation to Databases:** Many modern databases are hosted on cloud platforms, allowing access from anywhere and easy scaling.
- **Advantages:** Elastic storage, automated maintenance, high availability, reduced infrastructure costs.
- **Disadvantages:** Data privacy concerns, potential latency, vendor lock-in, and internet dependency.

Database Engines and Languages

- **What is a Database Engine?:** It is the underlying software that stores, retrieves, and manipulates data within a DBMS.
- **Popular Engines and Their Languages:**
 - **SQL Server:** Uses T-SQL
 - **Oracle:** Uses PL/SQL
 - **MySQL:** Uses ANSI SQL and procedural SQL extensions
 - **PostgreSQL:** Uses PL/pgSQL
- **Engine-Language Relationship:** Each engine has its own language features, but ANSI SQL offers a level of portability across systems.
- **Portability:** Some queries are compatible across engines; however, complex stored procedures and triggers may require adjustments.

Database Migration Between Engines

- **Feasibility:** Migration is possible between engines like SQL Server to MySQL or Oracle to PostgreSQL.
- **Challenges:**
 - Differences in data types (VARCHAR2 vs TEXT)
 - Incompatible SQL dialects

- Rewriting stored procedures, triggers, and constraints
- **Migration Considerations:**
 - Use migration tools (AWS DMS, Oracle SQL Developer)
 - Validate schema compatibility
 - Test for performance and data integrity post-migration

Logical vs Physical Schema

- **Logical Schema:** Abstract representation of the database structure including tables, fields, relationships, and constraints.
 - **Physical Schema:** Describes how data is actually stored in memory and disk, including file structures, indexing methods, and access paths.
 - **Differences:**
 - Logical: Focuses on design and user view.
 - Physical: Focuses on implementation and performance.
 - **Importance:** Understanding both allows efficient system design and performance optimization.
 - **Example:**
 - **Logical View:** Student table with attributes: ID, Name, Major
 - **Physical View:** Student table stored with indexing on ID, data files organized in disk blocks
-